

Entrega final Proyecto

Análisis y diseño de algoritmos

Índice

Definición del problema	2
Entradas	2
Salidas	3
Restricciones	3
Solución Fuerza bruta	4
Análisis de complejidad — Fuerza Bruta	4
Gráfica comparativa (tiempo vs tamaño de entrada).....	5
Análisis	5
Conclusión	5

Entrega 3 — Divide y Vencerás + Comparativa Final

Solución Divide y vencerás

Idea general del algoritmo

La estrategia de divide y vencerás aplicada a este problema consiste en dividir la lista de paquetes en subconjuntos más pequeños, resolver cada subconjunto de forma independiente y combinar los resultados en una única asignación final.

A diferencia de greedy o backtracking, este enfoque no evalúa el problema completo de una sola vez, sino que lo descompone hasta llegar a casos simples que pueden resolverse directamente.

Fases del problema

Fase 1 — Dividir

El algoritmo divide recursivamente la lista de paquetes en dos mitades iguales hasta llegar al caso base: una lista con un solo paquete.

Fase 2 — Vencer

Cuando la lista tiene un solo paquete, el algoritmo intenta asignarlo al primer camión donde quepa respetando la capacidad máxima. Si no cabe en ningún camión, el paquete se descarta.

Fase 3 — Combinar

Los resultados de ambas mitades se combinan automáticamente porque comparten las mismas estructuras de cargas y asignación. Cada mitad actualiza el estado de los camiones, y la siguiente mitad continúa desde ese estado.

Análisis de complejidad

Complejidad temporal

La relación de recurrencia del algoritmo es:

$$T(n) = 2T(n/2) + O(m)$$

Donde $O(m)$ representa el trabajo de asignar un paquete revisando hasta m camiones en el caso base.

Aplicando el Teorema Maestro con $a=2$, $b=2$ y $f(n)=O(m)$:

Como m es constante respecto a n , $f(n) = O(1)$ en términos de n , lo que da una complejidad de $O(n * m)$.

Complejidad espacial

El algoritmo genera una pila de llamadas recursivas de profundidad $\log(n)$, y en cada nivel almacena referencias a sublistas. La complejidad espacial es $O(n + \log n)$ que se simplifica a $O(n)$.

Medición empírica de tiempos

Caso	Paquetes	Camiones	Tiempo real
Pequeño	80	15	0.0000935 seg
Mediano	700	120	0.001338 seg
Grande	5000	900	0.05358 seg

Comparativa final

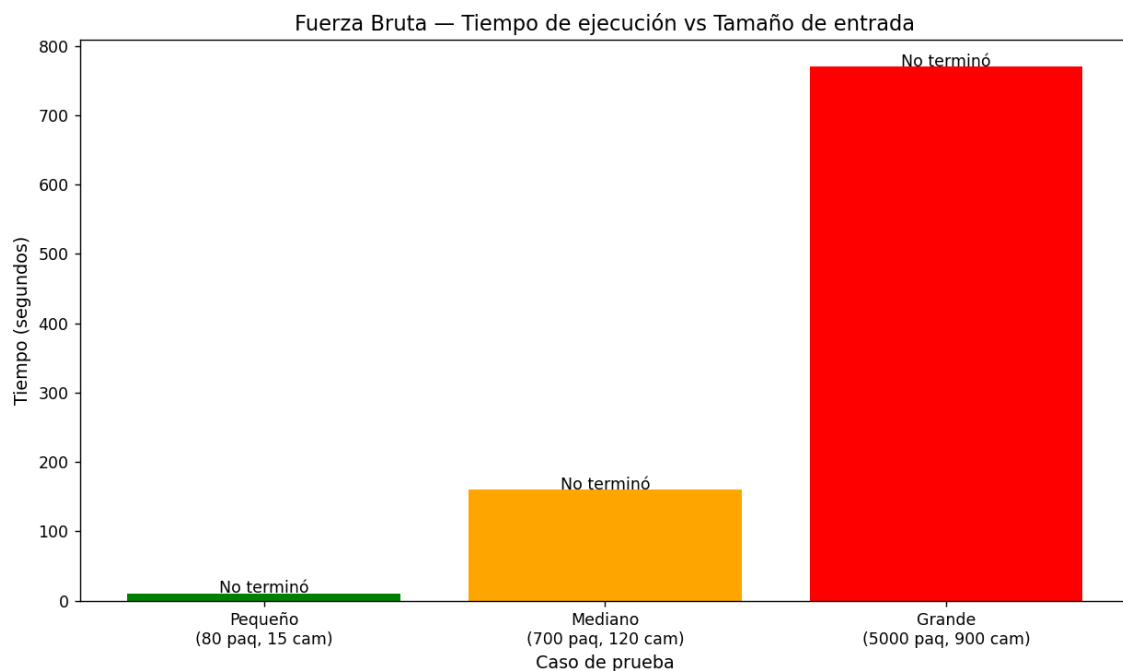
Tabla comparativa

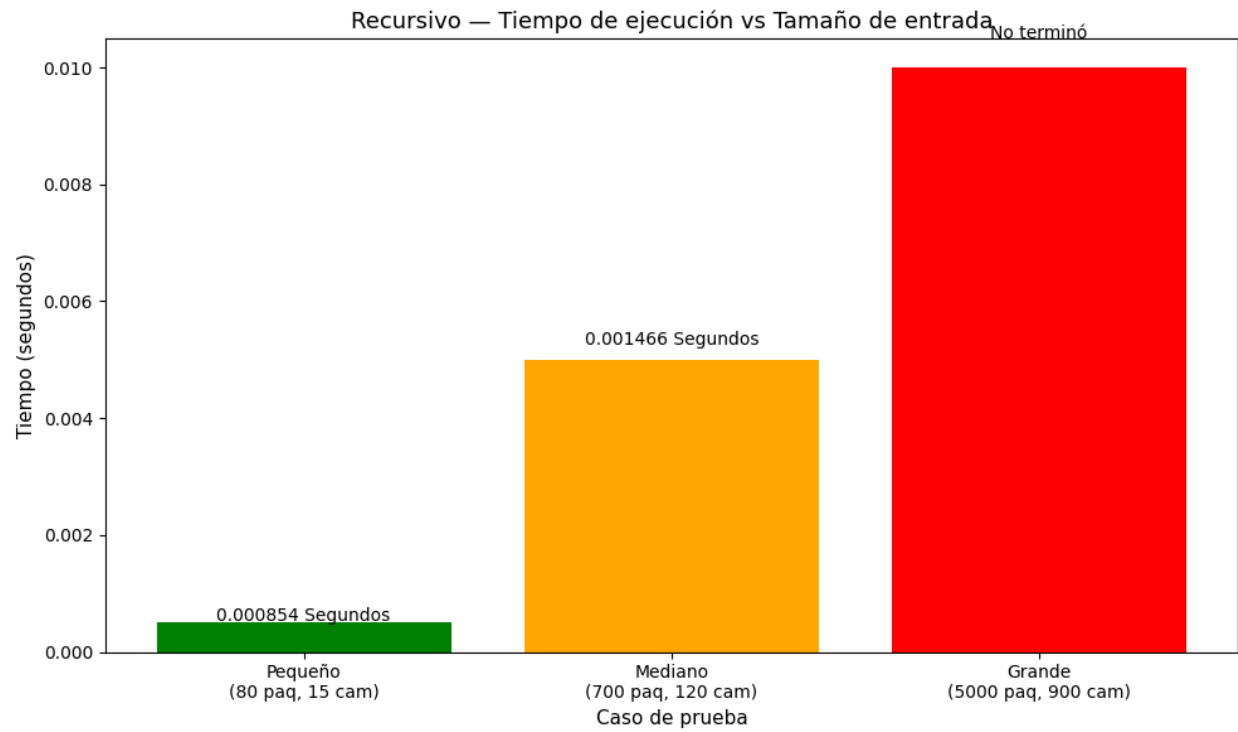
Técnica	Complejidad temporal	Complejidad espacial	¿Solución óptima?	Tiempo pequeño(80 paq, 15 cam)	Tiempo mediano(700 paq, 120 cam)	Tiempo grande(5000 paq, 900 cam)
Fuerza bruta	$O((m+1)^n)$	$O(n * m)$	Sí	No termina	No termina	No termina
Rekursiva	$O(n * m)$	$O(n)$	No	0.000854 seg	0.001466 seg	No termina
Greedy	$O(n * m)$	$O(n + m)$	No	0.000128 seg	0.001667 seg	No termina
Backtracking	$O((m+1)^n)$	$O(n * m)$	Sí	0.001530 seg	0.3733164 seg	No termina
Divide y Vencerás	$O(n * m)$	$O(n)$	No	0.0000935 seg	0.001338 seg	0.05358 seg

Gráficas de las soluciones

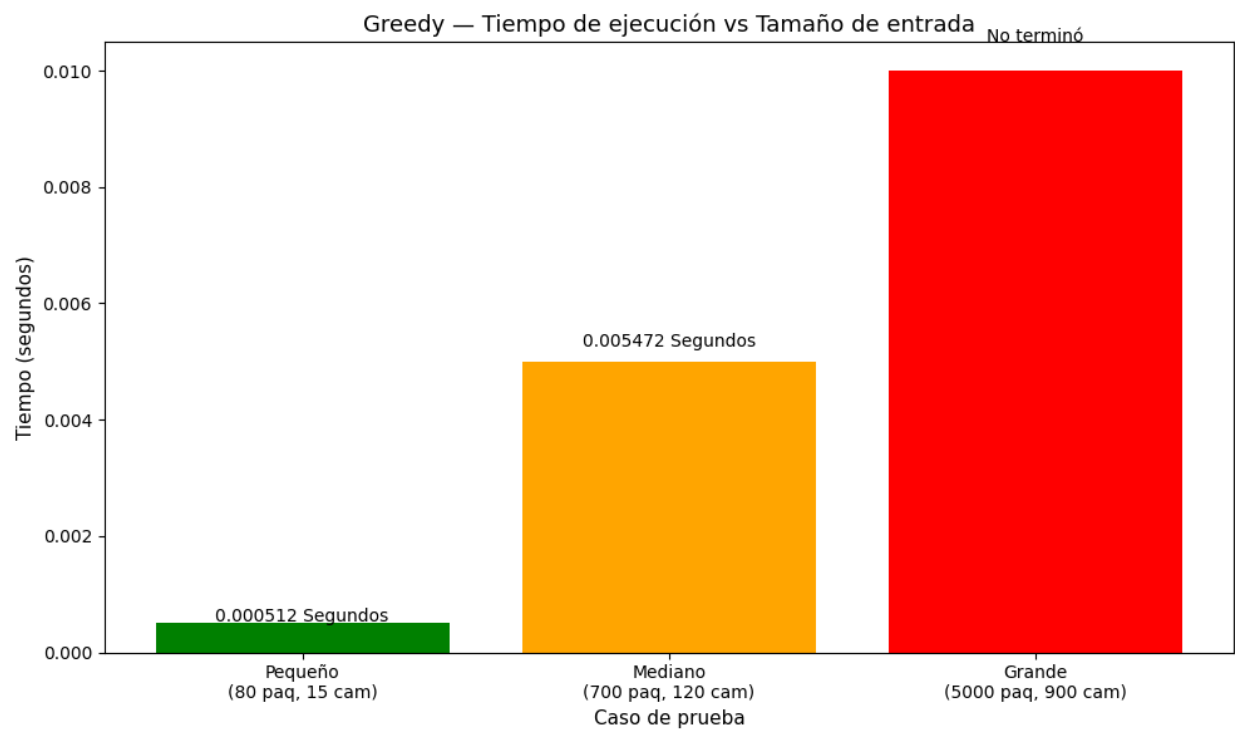
Fuerza bruta

Rekursiva

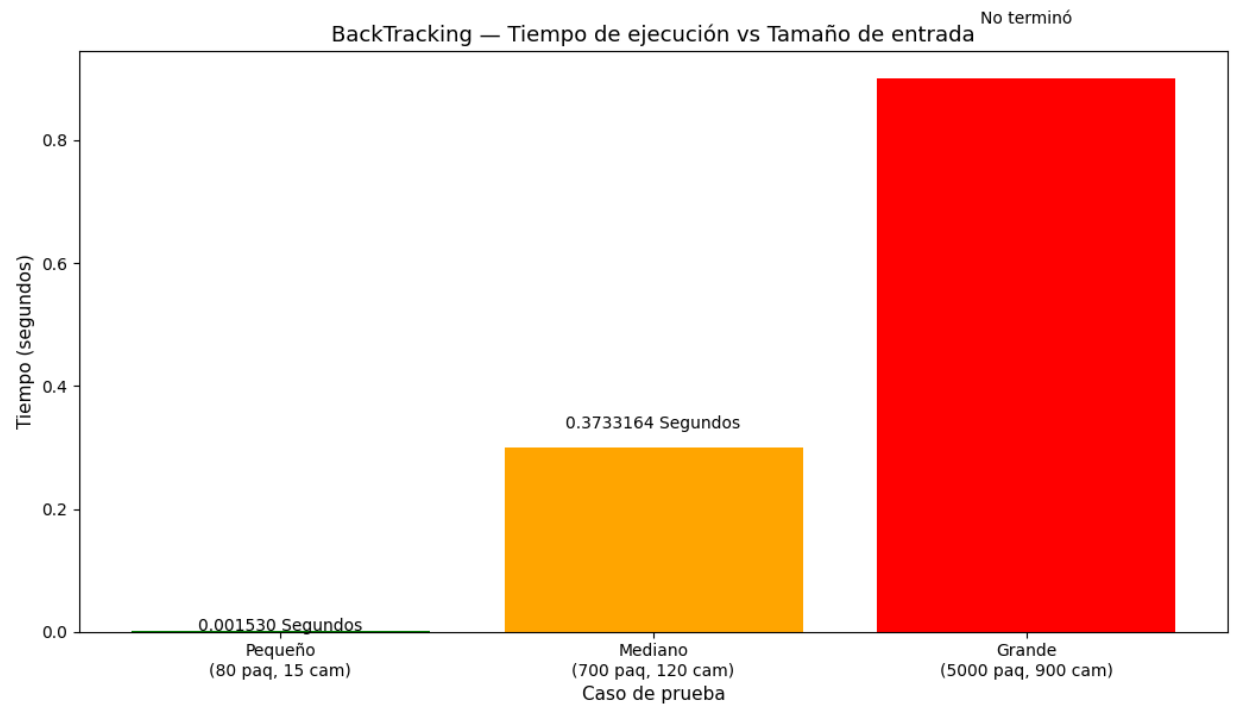




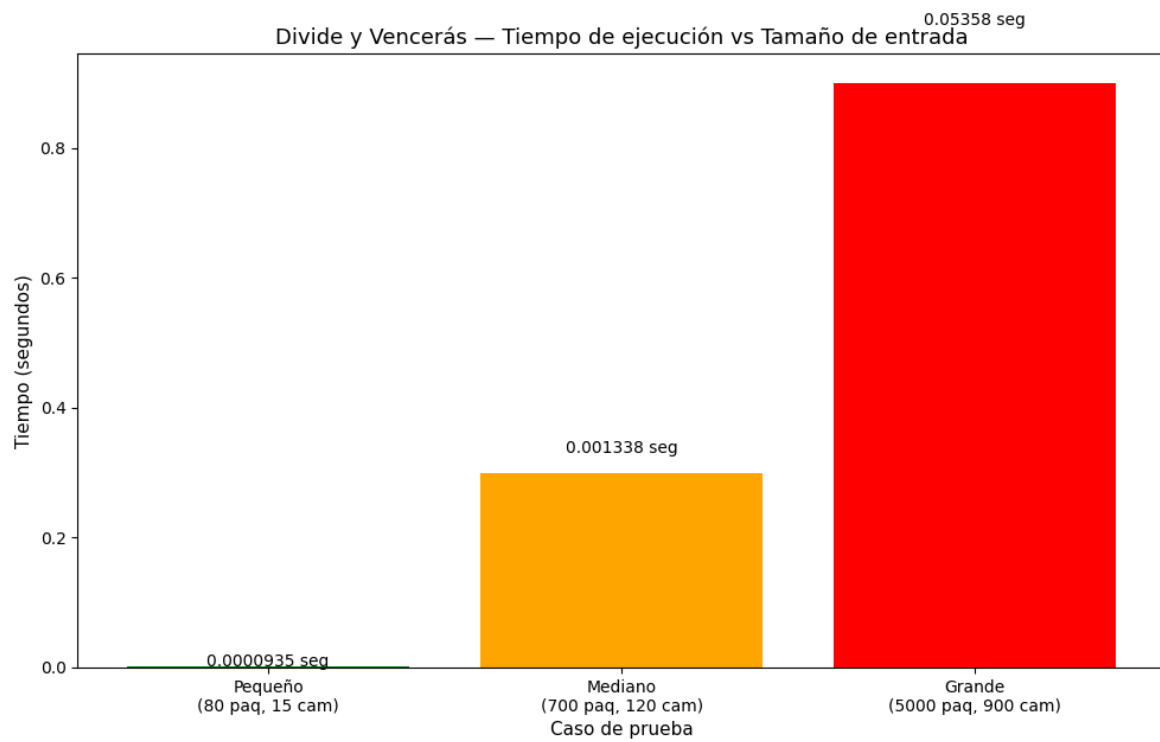
Greedy



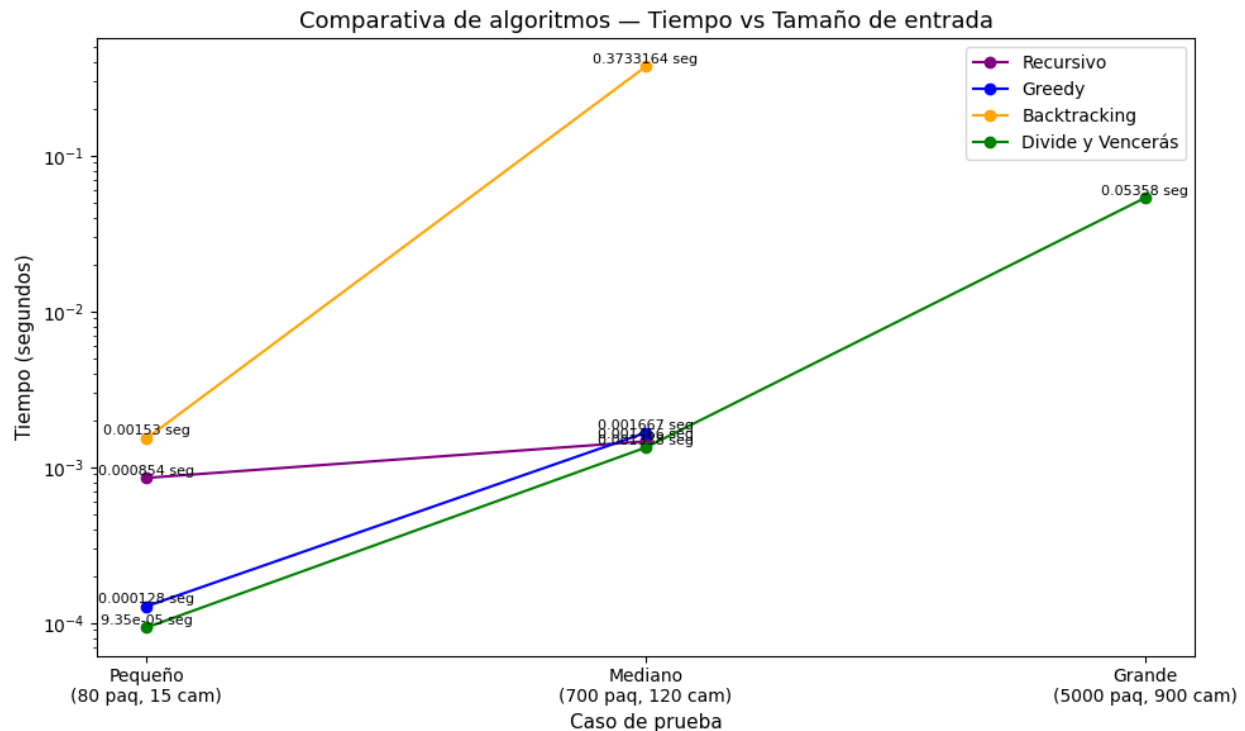
Backtracking



Divide y vencerás



General



Analisis

¿qué técnica es la más adecuada para este problema?

No existe una única técnica óptima para este problema, ya que la elección depende del contexto y las prioridades del sistema.

Pero podemos analizar los resultados obtenidos:

Divide y Vencerás es la técnica más adecuada para sistemas reales porque es el único algoritmo que logró resolver los tres casos incluyendo el grande (5000 paquetes, 900 camiones) en un tiempo razonable de 0.05358 segundos. Escala bien a medida que crece la entrada y su complejidad $O(n * m)$ es predecible y manejable.

Greedy es la segunda mejor opción cuando el tamaño de entrada es moderado. Su complejidad es igual a divide y vencerás pero no pudo resolver el caso grande, ya que las llamadas recursivas acumuladas consumen más memoria y tiempo de procesamiento del esperado, limitando su rendimiento en entradas muy grandes.

Backtracking es adecuado cuando la calidad de solución es crítica y el tamaño de entrada es pequeño o mediano. Garantiza la solución óptima pero su crecimiento exponencial lo hace inviable para entradas grandes.

Recursivo es útil cuando se necesita una solución rápida y válida sin importar la optimalidad. Es el más simple de implementar, pero falla en entradas grandes por el límite de llamadas recursivas.

Fuerza bruta queda descartada para cualquier uso práctico ya que no pudo resolver ninguno de los tres casos. Su único valor es académico, en este contexto podemos tomarlo como referencia para verificar que los demás algoritmos producen soluciones correctas.

A tener en cuenta

Cada algoritmo puede devolver una solución diferente porque tienen estrategias distintas:

Fuerza bruta: solución óptima (la mejor posible)

Recursivo: primera opción válida encontrada, sin criterio de selección

Greedy: buena solución basada en prioridad, pero no garantiza el óptimo

Backtracking: solución óptima igual que fuerza bruta, pero más rápido gracias a la poda

Divide y vencerás: depende de cómo se divida, puede variar.

Conclusiones

El desarrollo de este proyecto permitió analizar y comparar diferentes técnicas algorítmicas para resolver el problema de asignación de paquetes a camiones, evidenciando cómo la complejidad de un algoritmo influye directamente en su rendimiento y viabilidad práctica. A través de las pruebas realizadas, se observó que los algoritmos de complejidad exponencial, como fuerza bruta y back tracking, aunque garantizan soluciones óptimas, presentan tiempos de ejecución demasiado altos para manejar entradas grandes, lo que limita considerablemente su uso en escenarios reales.

Por otro lado, las técnicas recursivas, greedy y divide y vencerás demostraron un mejor desempeño en términos de eficiencia y escalabilidad. En especial, divide y vencerás fue la solución más estable y eficiente, ya que logró resolver todos los casos de prueba, incluyendo el caso grande de 5000 paquetes y 900 camiones, manteniendo tiempos de ejecución bajos y consistentes. Esto demuestra que los algoritmos con complejidad $O(n^*)$

m)) son mucho más adecuados para aplicaciones que requieren procesar grandes volúmenes de información en tiempos razonables.

Finalmente, este proyecto permitió comprender que no existe una única técnica ideal para todos los problemas, sino que la elección del algoritmo depende de las necesidades específicas del sistema. Si se busca la solución óptima, algoritmos como backtracking pueden ser útiles en entradas pequeñas; mientras que, si se prioriza velocidad y capacidad de escalamiento, técnicas como divide y vencerás o greedy resultan más apropiadas para entornos reales.